

When Should StorFuzz Switch? Studying Diversification Interval Length and Runtime Signals

Riley Immel

University of Virginia
Charlottesville, Virginia, USA
immel@virginia.edu

1 Introduction

StorFuzz is a fuzzing technique introduced by Weiß et al. that attempts to help fuzzing campaigns escape code coverage plateaus by tracking data coverage, the values written by store instructions during program execution. In their paper, they show that alternating between 24 hours of conventional fuzzing (with LibAFL) and 24 hours of diversification with their StorFuzz implementation, resulted in significant coverage improvements compared to continuously running LibAFL with no diversification.

The 24-hour choice is somewhat arbitrary, though, and that is what motivated the question I tried to answer with this project: how long should diversification actually run before a campaign switches back to LibAFL from StorFuzz?

2 Background & Motivation

As mentioned, StorFuzz makes use of data coverage in fuzzing as opposed to a less granular method such as edge coverage. Edge coverage tracks which control-flow transitions between basic blocks have been exercised by at least one input. It is the standard feedback signal used by coverage-guided fuzzers such as AFL and LibAFL. Data coverage instead records the values written by store instructions, so two inputs that hit the same edges but produce different internal program states are treated as distinct. The key insight from the StorFuzz paper is that by diversifying the fuzzing corpus with inputs that produce different internal states, even when the code coverage stays the same (as measured by edge coverage), conventional fuzzers are able to "escape" their plateaus and reach new code regions [1].

One part of their experiment that was not grounded in any analysis, however, was the authors' decision to use 24-hour periods of diversification. They explicitly state that "[f]uture work should ... investigate for how long StorFuzz needs to run for a sufficient amount of diversity" [1].

This arbitrary choice motivated the project. Should the switch back occur before, after, or at 24 hours, and is there a way to determine the right switch point at run-time rather than fixing it in advance? The diversification period length has real practical implications, since fuzzing is a resource-intensive process and compute time is valuable.

It is worth being explicit about why I treat edge coverage as the goal here, since data coverage is StorFuzz's own feedback signal and it would be easy to assume that more data coverage is automatically good. The point of fuzzing is to find bugs, and the standard proxy for that is code coverage: a bug can only trigger on code that actually runs, so reaching more code (more edges) gives the campaign more chances to hit one. Data coverage is not a competing goal in this picture, it is a means to that end. StorFuzz uses data coverage as

feedback because it is a finer-grained signal that can pull a fuzzer off an edge-coverage plateau, but what the campaign is ultimately judged on is still how much code it reaches. A corpus that is more diverse in data coverage but does not unlock new edges is one that captures different internal views of the same reachable code, which does not directly help find new bugs. That is why edge coverage is the bottom-line metric throughout this paper, and it is also the metric the original StorFuzz paper evaluated against.

To address the open question from the StorFuzz paper, I evaluated three diversification periods, 2, 6, and 24 hours, followed by a fixed 12-hour LibAFL phase on two benchmarks from the StorFuzz paper: OpenThread and HarfBuzz.

3 Experimental Setup

To evaluate the effect of diversification period length, I ran StorFuzz using three different diversification periods: 2 hours, 6 hours, and 24 hours. After each diversification period, the resulting corpus was passed to LibAFL, which was then run for a fixed 12-hour period. I refer to these configurations as 2+12, 6+12, and 24+12 respectively.

I ran these configurations on two benchmarks from the StorFuzz paper: HarfBuzz and OpenThread. For each benchmark and each diversification period, I ran 9 independent trials. This resulted in 54 total trials: 27 for HarfBuzz and 27 for OpenThread. Each benchmark had 9 trials for each of the three diversification periods.

Both benchmarks were started from the seed corpora that ship with the StorFuzz evaluation, which come directly from each target's existing fuzz harness: the hb-shape-fuzzer corpus for HarfBuzz and the matching harness corpus for OpenThread. I used the same seeds as the original paper so the inputs would be comparable to its setup, even though my diversification schedule and total runtime are different.

Each trial consisted of two phases. In Phase 1, StorFuzz ran in diversification mode for the selected diversification period. In Phase 2, LibAFL ran for 12 hours using the corpus produced by the diversification phase. This setup allowed me to compare both the direct coverage obtained during diversification and the additional coverage LibAFL was able to find after switching.

The primary metric used was edge coverage, measured as the number of edges hit over time. This was chosen because the main question is whether different diversification periods help the campaign reach more code, and because this was also the metric used in the original paper. I also tracked the number of new edges found during Phase 2, since this indicates how useful the diversified corpus was once LibAFL resumed. In addition, I used data coverage and corpus growth during the diversification phase as possible runtime signals for deciding when to switch.

Table 1: Median results across 9 trials for each benchmark and diversification period. Gain/hr is total edge gain per hour after ignoring the first 10 minutes.

Benchmark	DP	Edges			Gain/hr
		Final	14h	Phase 2	
HarfBuzz	2h	4763	4763	236	28.49
HarfBuzz	6h	4777	4761	129	22.89
HarfBuzz	24h	4786	4697	51	11.47
OpenThread	2h	4765	4765	34	7.81
OpenThread	6h	4777	4772	24	5.95
OpenThread	24h	4780	4764	4	3.04

For analysis, I compared the final edge coverage of each configuration, the edge coverage reached under a common 14-hour time budget, and the total edge gain per hour. The common-budget comparison is important because the three configurations have different total runtimes: 14 hours for 2+12, 18 hours for 6+12, and 36 hours for 24+12. Without this comparison, the 24-hour diversification period has a much larger time budget than the shorter configurations.

To avoid the initial seed-corpus loading behavior dominating the measurements, I ignored the first 10 minutes of each run when calculating coverage gains and rates. The 10-minute cutoff was chosen by inspecting the per-trial logs: in every trial, the steep startup ramp from loading and re-executing the seed corpus had completed within the first few minutes, after which the rate curves settled into a much slower growth-rate. Ten minutes was the smallest round-number window that consistently fell after this transition across both benchmarks, so it provides a conservative buffer without discarding meaningful sustained-phase activity. This made the comparisons focus more on the sustained behavior of each configuration rather than the initial burst of coverage at startup.

4 Results

Figure 1 shows the median edge-coverage trajectory for each configuration. On both benchmarks, the curves rise steeply during the first hour and then begin to slow down/flatten. The 24+12 trajectory keeps climbing only very gradually after the 2- and 6-hour switch points have already passed. This is the same diminishing-returns pattern that the summary statistics quantify below.

The results show two main trends. First, the 24-hour diversification period generally reached the highest final edge coverage, but only by a small margin. Second, the shorter diversification periods were much more efficient when accounting for the amount of time spent fuzzing. Table 1 summarizes the main median results across the 9 trials for each configuration.

4.1 Final Edge Coverage

Figure 2 shows the final edge coverage for each configuration. On both benchmarks, the 24+12 configuration had the highest median final edge coverage.

This means that longer diversification did improve final coverage, but the improvement was relatively small. Compared to the 2-hour

diversification period, the 24-hour diversification period improved median final coverage by 23 edges on HarfBuzz and 15 edges on OpenThread. Compared to the 6-hour diversification period, the improvement was even smaller: 9 edges on HarfBuzz and 3 edges on OpenThread.

4.2 Common-Budget Comparison

Since each configuration has a different total runtime, final edge coverage alone does not tell the full story. The 2+12 configuration runs for 14 total hours, the 6+12 configuration runs for 18 total hours, and the 24+12 configuration runs for 36 total hours. Because of this, I also compared edge coverage at a common 14-hour wall-clock budget.

This comparison is shown in Figure 3. Under this common budget, the 24-hour diversification period is no longer the best configuration. On HarfBuzz, the 2+12 and 6+12 configurations were very close, while the 24+12 configuration was behind because it had not yet switched back to LibAFL. On OpenThread, the 6+12 configuration had the highest median edge coverage at 14 hours.

This result is important because the 24-hour configuration has not even switched back to LibAFL by the 14-hour mark. While it eventually reaches slightly higher final coverage, it does so by using much more time. In a compute-limited setting, this makes the shorter diversification periods more attractive.

4.3 Coverage Efficiency

To compare the configurations in terms of efficiency, I calculated the total edge gain per hour after ignoring the first 10 minutes of each run. This is shown in Figure 4. By this metric, the 2-hour diversification period was the best configuration on both benchmarks.

As shown in Table 1, the median gain per hour decreased as the diversification period increased. This suggests that the additional coverage found by the 24-hour configuration came at a high cost. The 24-hour diversification period did continue to make progress, but its rate of useful coverage discovery was much lower than the shorter configurations.

4.4 Coverage After Switching Back to LibAFL

I also measured how many new edges were found during the 12-hour LibAFL phase after diversification. This is shown in Figure 5. The shorter diversification periods had much larger gains during the LibAFL phase.

This made me wonder whether longer diversification was actually building a richer corpus, or just adding more inputs that map to the same internal states the shorter runs already saw. I checked by going back through the trial logs.

On HarfBuzz, going from 2 hours to 24 hours of diversification grows the median Phase-1 corpus from about 13,600 to 19,200 inputs, a 42% increase, and the median data-coverage count from about 41,700 to 49,600 distinct internal states, a 19% increase. But the median Phase-1 edge count only rises from 4,520 to 4,736, and the final edge count after Phase 2 only rises from 4,763 to 4,786, a 0.5% increase. OpenThread shows the same pattern in a more compressed form: the Phase-1 corpus grows from about 2,200 to 3,200 inputs and data coverage from 46,400 to 48,100 entries between 2-hour and 24-hour diversification, while final edge coverage moves from 4,765

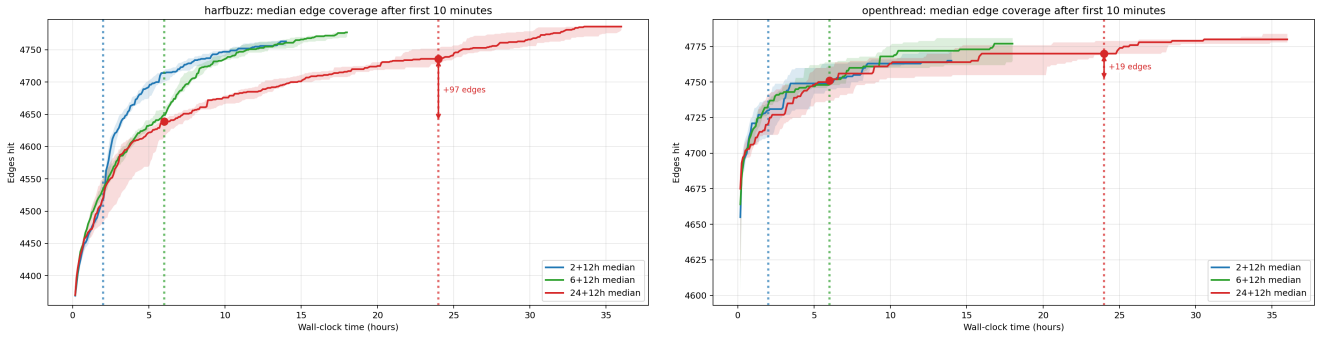


Figure 1: Median edge coverage over time across the 9 trials per configuration, with shaded interquartile range, excluding the first 10 minutes to remove the seed-corpus startup ramp. Dashed vertical lines mark the diversification-to-LibAFL switch for each configuration. The 2+12 and 6+12 trajectories plateau well before their respective switch points, and the 24+12 curve makes only marginal additional progress over the long tail of its diversification phase.

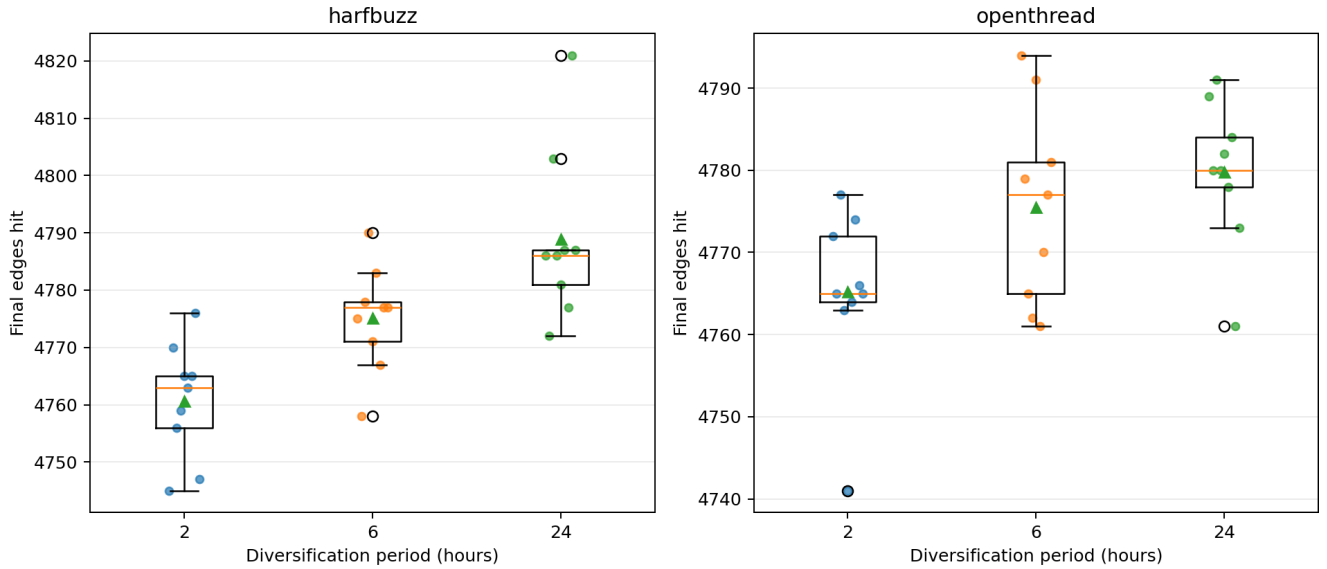


Figure 2: Final edge coverage for each diversification period. The 24-hour configuration has the highest median final coverage, but the absolute difference is small.

to 4,780. The long-diversification corpus really is more diverse by StorFuzz’s own metric. It contains thousands of additional inputs that exercise distinct internal states. But most of those extra states are reached through edges that have already been discovered, so they are novel as data coverage and redundant as edge coverage.

This reframes the Phase-2 gap. The 2+12 configuration is not finding 236 edges in Phase 2 that 24+12 misses. Those are largely the same edges that 24+12 finds late in its diversification phase, just discovered at LibAFL’s faster execution rate rather than under StorFuzz’s heavier instrumentation. The 24-hour diversification period ends up doing a job that LibAFL could do faster. The small final-edge advantage of 24+12 looks more like evidence that StorFuzz only needs a few hours to build a corpus diverse enough for LibAFL

to use. Beyond that point, extra diversification mostly produces redundant data-coverage entries rather than new reachable code.

5 Runtime Signal Analysis

The second question I considered was whether runtime metrics during the diversification phase could help indicate when StorFuzz should switch back to LibAFL. For this analysis, I focused on two signals from the diversification phase: the rate of new data coverage discoveries and the rate of corpus growth. These were chosen because they directly relate to StorFuzz’s immediate job during Phase 1, which is to build a more diverse corpus for LibAFL to use, rather than to chase edge coverage on its own. StorFuzz is not meant to be run by itself; it is meant to feed a regular fuzzer like LibAFL with

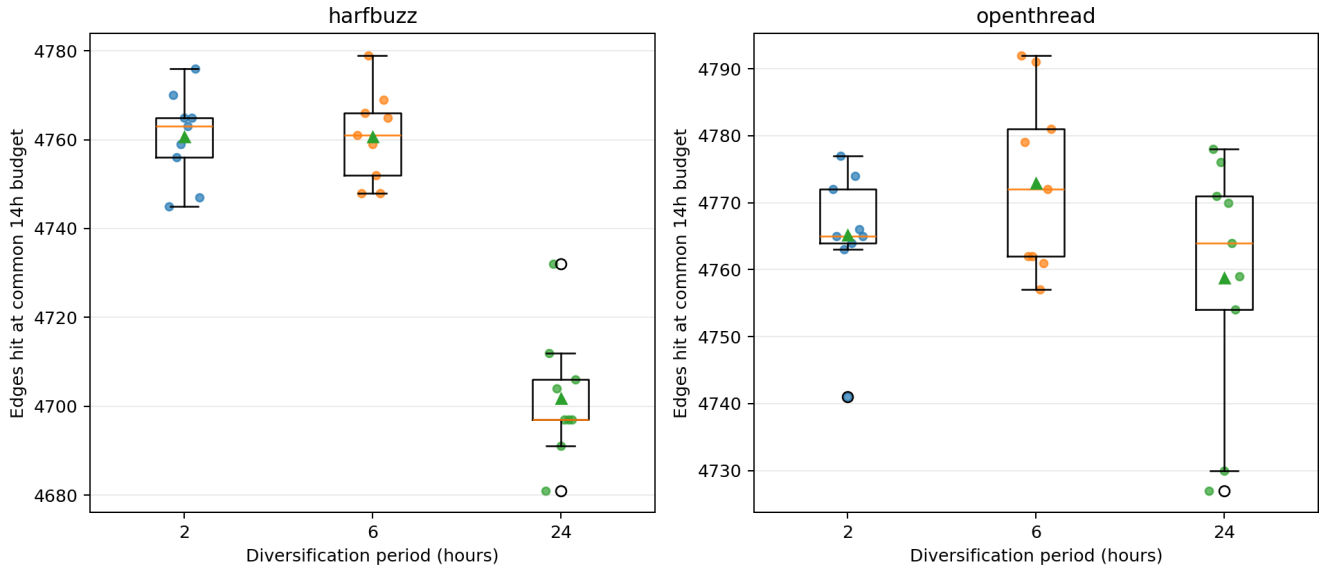


Figure 3: Edge coverage at a common 14-hour wall-clock budget. This comparison controls for the fact that the full configurations have different total runtimes.

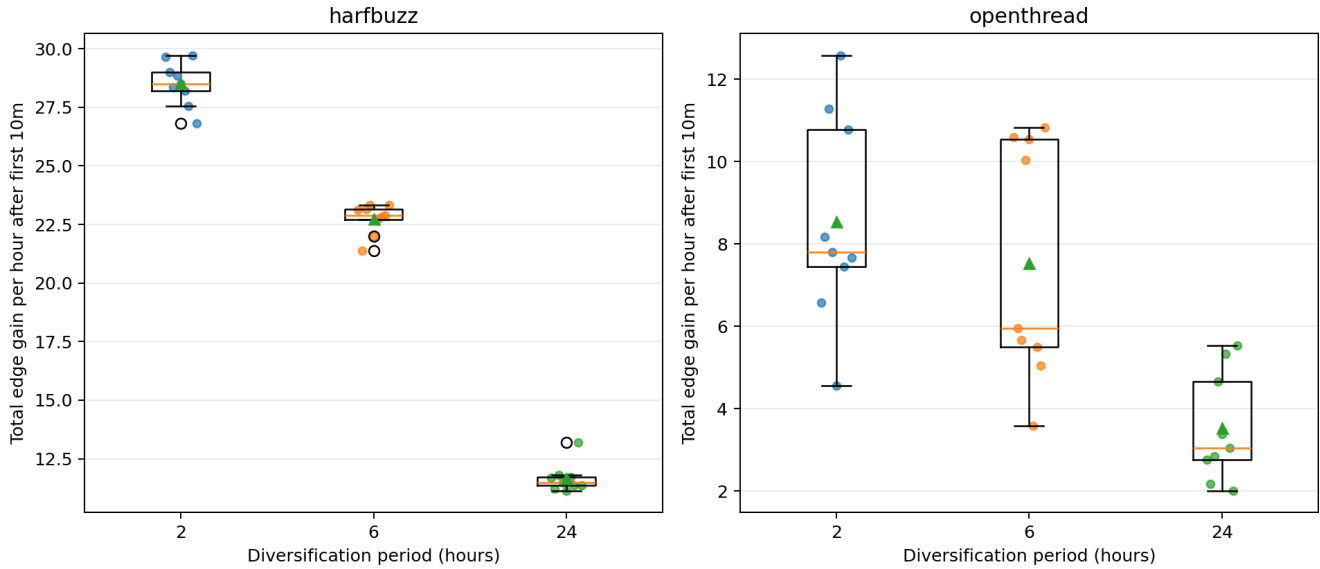


Figure 4: Total edge gain per hour for each configuration. The 2-hour diversification period was the most efficient on both benchmarks.

a corpus that exercises a wider range of internal program states, and it is then up to LibAFL to turn that diversity into new edges. So data-discovery and corpus-growth rates are the right runtime signals for telling whether Phase 1 still has useful work left to do, even though edge coverage is still the ultimate metric the overall campaign is judged on.

For each trial, I measured the data coverage discovery rate and corpus growth rate during the final hour of the diversification phase. I then compared these values against the number of new edges found during the following 12-hour LibAFL phase. The intuition was that if the diversification phase was still finding many new data states near the end, then switching at that point might still give LibAFL a useful corpus to build from. On the other hand, if data

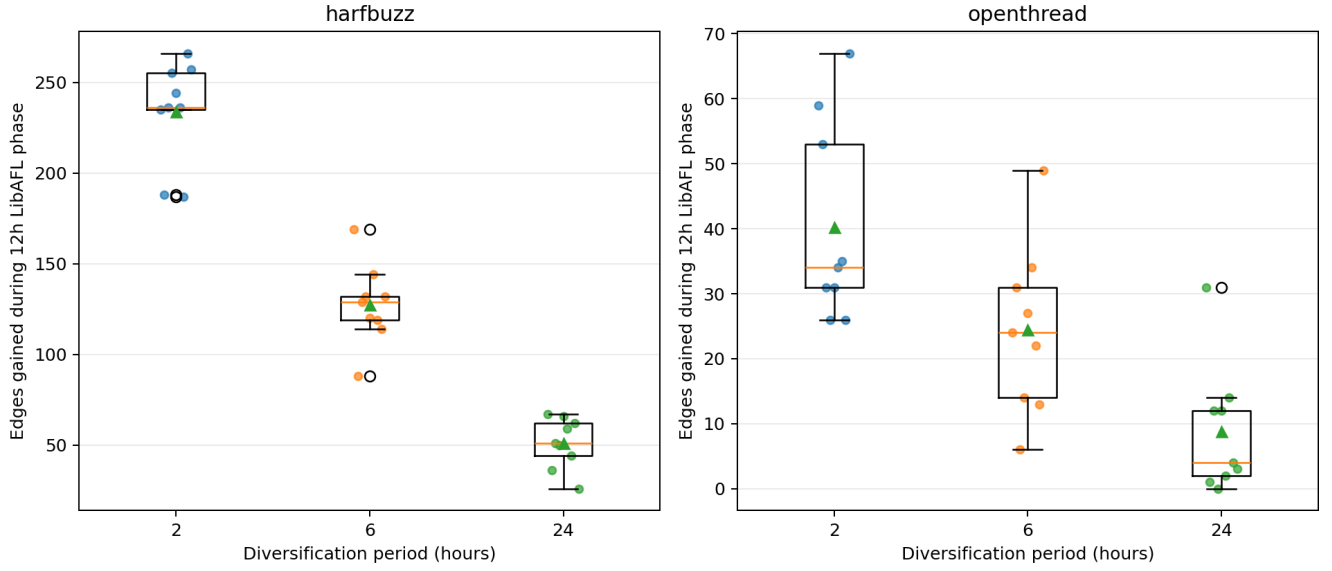


Figure 5: Edges gained during the 12-hour LibAFL phase. Shorter diversification periods leave more remaining coverage for LibAFL to discover after switching.

Table 2: Spearman correlation between final-hour diversification signals and Phase-2 edge gain.

Benchmark	Signal	Spearman r
HarfBuzz	Data discoveries/hour	0.910
HarfBuzz	Corpus additions/hour	0.877
OpenThread	Data discoveries/hour	0.704
OpenThread	Corpus additions/hour	0.705

coverage and corpus growth had mostly flattened out, this could suggest that diversification was reaching diminishing returns.

Figure 6 shows this relationship. For HarfBuzz, both runtime signals were strongly correlated with the number of edges found after switching back to LibAFL. The Spearman correlation between final-hour data discoveries and Phase-2 edge gain was 0.910, and the correlation for final-hour corpus growth was 0.877. OpenThread showed the same general trend, but the relationship was weaker, with correlations of 0.704 for data discoveries and 0.705 for corpus growth.

These results suggest that runtime signals are useful, but they should be interpreted carefully. The correlations are partly affected by the experimental setup because the diversification period length was intentionally varied. Shorter diversification periods tend to have higher final-hour discovery rates simply because they are earlier in the campaign, while longer diversification periods tend to have lower rates because they have had more time to approach a plateau.

Because of this, I do not think these results justify a single fixed threshold such as switching whenever data discoveries fall below some absolute number. A threshold that works for HarfBuzz may

Table 3: Retrospective adaptive switch times on the 24+12 trials. Each trial’s switch time is the first point at which the 30-minute rolling data-discovery rate drops below a given fraction of that trial’s post-warmup peak rate. Medians across 9 trials, with min–max ranges.

Benchmark	Threshold	Median (h)	Range (h)
HarfBuzz	50% of peak	1.01	0.80–1.54
HarfBuzz	25% of peak	1.90	1.11–2.55
HarfBuzz	10% of peak	3.33	2.57–4.96
OpenThread	50% of peak	1.00	0.80–1.99
OpenThread	25% of peak	1.36	1.21–2.35
OpenThread	10% of peak	2.58	1.43–3.99

not work for OpenThread, and the raw rates differ substantially between the two benchmarks. Instead, the results point toward a per-trial rule: watch the data-coverage or corpus-growth rate against that run’s own earlier behavior, and switch once the rate has dropped substantially. The rest of this section tries that rule out retrospectively on the trial logs.

I ran a post-hoc simulation on the nine 24+12 trials per benchmark, since those are the only trials in which diversification runs long enough to see a sustained plateau. For each trial I computed a 30-minute rolling rate of new data-coverage discoveries throughout Phase 1, recorded the peak rate after the 10-minute warmup window, and found the first time the rolling rate dropped below a fixed fraction of that peak. Each trial defines its own baseline, so the rule does not depend on absolute discovery rates that vary between targets. Table 3 reports the resulting switch times across three threshold settings.

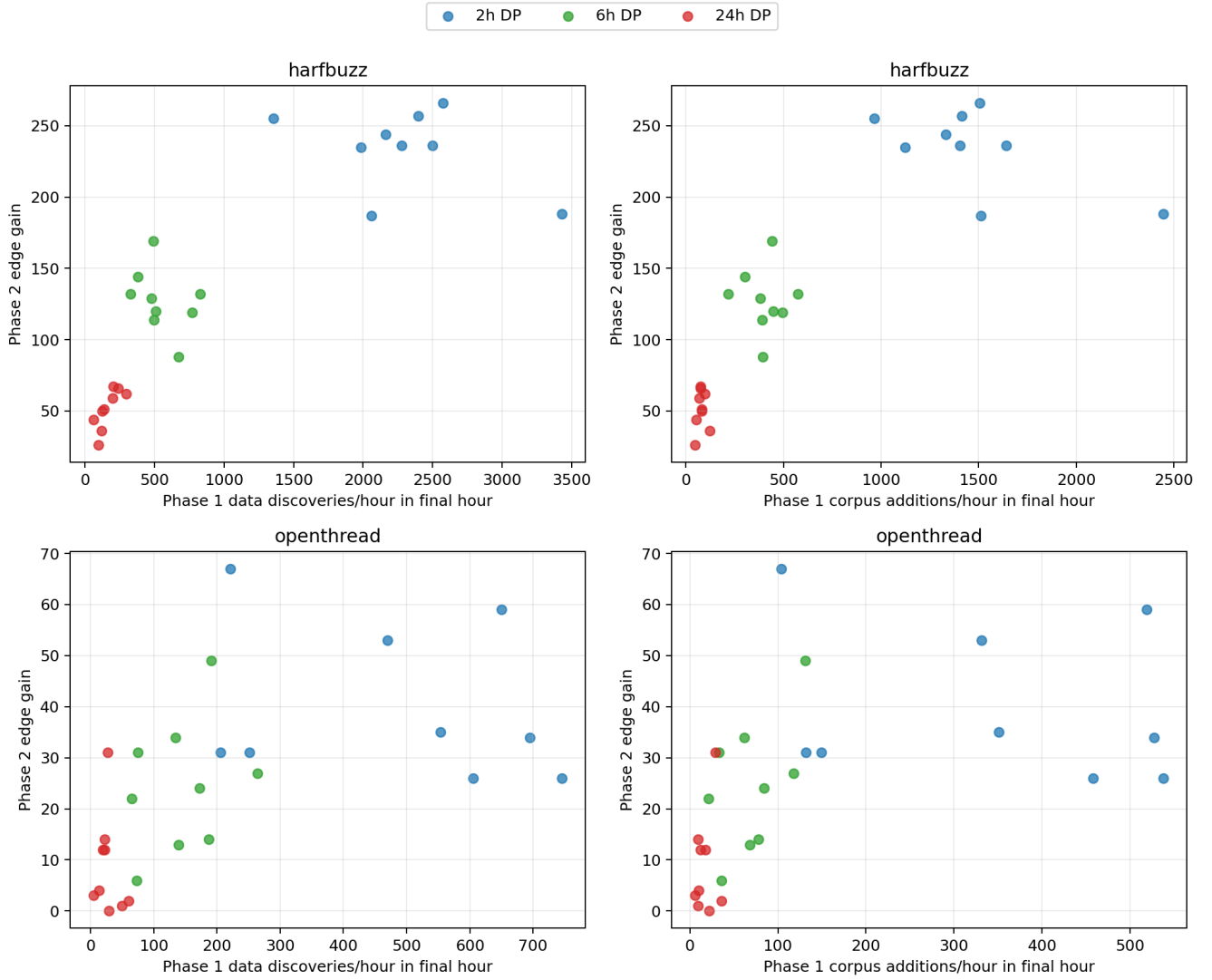


Figure 6: Relationship between final-hour diversification activity and edge gain during the following LibAFL phase. Higher data-discovery and corpus-growth rates generally correspond to larger Phase-2 gains, especially on HarfBuzz.

Two things stand out. First, even the most permissive threshold tested, waiting until the discovery rate has fallen to 10% of its peak, fires within 5 hours on every trial and within roughly 3 hours at the median on both benchmarks. No trial comes close to justifying a 24-hour interval. The rolling rate has already collapsed by an order of magnitude long before that. Second, the 25% threshold places the median switch at 1.9 hours on HarfBuzz and 1.4 hours on OpenThread, which closely tracks the finding that the 2+12 configuration is the most efficient fixed schedule. The adaptive rule recovers the same conclusion as the fixed-schedule comparison without having to be told the right interval in advance.

The simulation has two clear caveats; the first of which is that it is retrospective. Each trial’s switch time is computed from its own logged Phase-1 data, so this is not a closed-loop evaluation

of an adaptive policy actually controlling a fuzzing campaign. The second being that it also cannot directly measure the Phase-2 edge gain that an adaptive switch would have produced for an arbitrary cutoff, because Phase 2 in each trial was only run from that trial’s fixed cutoff (2, 6, or 24 hours), not from a corpus snapshot taken at an adaptive switch time inside a longer run. The 2+12 trials do offer a useful proxy for the 25% threshold, since their fixed 2-hour cutoff sits very close to that threshold’s median switch time. The corresponding Phase-2 edge gains, a median of 236 edges on HarfBuzz and 34 on OpenThread, are a reasonable approximation of what an adaptive 25% policy would produce on these benchmarks. What the simulation does establish is a lower bound on how much diversification time is needed before StorFuzz’s own data-coverage signal indicates diminishing returns. Across both benchmarks and

across reasonable threshold choices, that point arrives well before the 24-hour mark and lines up with the 2–6 hour range that performed best in the fixed-schedule experiments. A natural next step is to implement this rule inside StorFuzz and evaluate the resulting campaigns directly.

6 Discussion

The Phase-2 gap between the configurations is not really evidence that short diversification produces a better corpus. It looks more like the same coverage shifted around. As I showed in Section 4.4, longer diversification grows the corpus and data coverage by a lot (the HarfBuzz Phase-1 corpus grows 42% from the 2-hour to the 24-hour configuration), but final edge coverage only goes up by 0.5%. The extra inputs are novel by data coverage and redundant by edge coverage. The 24-hour run ends up doing work that LibAFL could do faster, just with heavier instrumentation in the way.

The fixed-schedule comparison and the runtime-signal simulation end up pointing in the same direction. The 2+12 and 6+12 configurations get most of the edge-coverage benefit available on these targets, and the self-relative threshold rule (Section 5, Table 3) lands in the same 1–3 hour window per trial without me having to pick an interval up front. Two different analyses agreeing on roughly the same range is, I think, the strongest signal in this study that 24 hours is more diversification than these benchmarks need.

For practical use on similar targets, a 2-hour fixed schedule is the most efficient option I tested, and 6 hours is a reasonable middle ground if final coverage matters more than runtime cost. The longer-term direction is to drop the fixed schedule entirely and let StorFuzz decide based on its own discovery curve.

7 Threats to Validity / Limitations

The most important limitation of this study is the absence of a pure LibAFL-only baseline. The goal was to compare different StorFuzz diversification lengths against each other under the same local setup, not to re-evaluate whether StorFuzz improves over LibAFL alone. As a result, these results can support conclusions about whether 2, 6, or 24 hours of diversification is more efficient than the others, but they cannot independently show that any of these schedules is better than running LibAFL continuously in this environment. Any claim that short diversification “captures most of the benefit of StorFuzz” is a claim relative to longer diversification, not relative to no diversification at all.

The second important limitation is the size of the benchmark set. Only two targets, HarfBuzz and OpenThread, were evaluated. Both come from the StorFuzz paper, but two programs are not enough to make a universal claim about the best diversification period. Targets with different input formats, state spaces, or coverage-plateau structures could respond differently to diversification. The recommendation that 2–6 hours is generally sufficient should be treated as suggestive until tested on more programs.

The runtime-signal results in Section 5 should also be treated as exploratory rather than prescriptive. The Spearman correlations between final-hour diversification activity and Phase-2 edge gain are real, but the correlation strength is partly a consequence of the experimental design. Shorter diversification periods are observed

at earlier points in their own discovery curves, so they are guaranteed to have higher final-hour rates than longer periods. The retrospective adaptive-switch simulation partly addresses this by using each trial’s own peak rate as a self-relative reference, but it is still a post-hoc analysis of fixed-schedule trials rather than a closed-loop evaluation of an adaptive policy.

The remaining limitations are more procedural. The fuzzing was run on a virtual machine with 16 vCPUs, 16 GB of RAM, and 40 GB of disk, with at most 3 trials in parallel and per-instance core pinning. This is a smaller and more constrained environment than the hardware used in the original StorFuzz evaluation. The Phase-2 LibAFL window was fixed at 12 hours rather than the 24 hours used in the original paper, so these results should not be read as a reproduction of that work but as a smaller-scale comparison under a consistent local setup. Finally, the three configurations have different total runtimes (14, 18, and 36 hours). This is addressed by the common 14-hour budget comparison in Figure 3, but it is still a complication when interpreting final-coverage differences.

8 Conclusion

This project evaluated how long StorFuzz should run in diversification mode before switching back to LibAFL. I tested 2-hour, 6-hour, and 24-hour diversification periods on HarfBuzz and OpenThread, with each diversification phase followed by a fixed 12-hour LibAFL phase.

The 24-hour configuration reached the highest median final edge coverage on both benchmarks, but only by a small margin (9 median edges on HarfBuzz and 3 on OpenThread over the 6-hour configuration). The 2-hour configuration was the most efficient by edge gain per hour on both benchmarks. A retrospective simulation of a self-relative threshold rule (Section 5) reached the same recommendation: the adaptive switch fires in the 1–3 hour range across both targets and every threshold I tested. On these benchmarks, 24 hours is more diversification than is needed.

These conclusions are all relative to longer fixed diversification periods. I did not run a LibAFL-only baseline in this environment, so I cannot claim that any of these schedules beats running LibAFL continuously. Future work should test the recommended range on more benchmarks, run a LibAFL-only control for a direct comparison, and implement the self-relative switching rule inside StorFuzz so that it can be evaluated as an actual policy rather than after the fact.

Artifact Availability

The code, processed trial data, and plots are available at <https://github.com/RileyImmel/storfuzz-diversification-interval>. The raw per-trial fuzzing log data is about 86 GB and is thus not included in the repository. All figures, numbers, and the table in this paper are able to be reproduced using just the CSV summaries that are included in the `analysis/` directory.

References

- [1] Leon Weiß, Tobias Holl, and Kevin Borgolte. 2026. StorFuzz: Using Data Diversity to Overcome Fuzzing Plateaus. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)* (Rio de Janeiro, Brazil, 2026-04), Mira Mezini and Thomas Zimmermann (Eds.), Vol. 48. Association for Computing Machinery (ACM)/Institute of Electrical and Electronics Engineers (IEEE). doi:10.1145/3744916.3773179